

Homework 3

ASEN 5090 Introduction to GNSS

Justin Le

September 25, 2026

Problem 1: GPS Observations

(a.)

% Read rinex file`rinexData = rinexread('NIST00USA_R_20262310000_01D_30S_MO.rnx');``GPS_rinexData = rinexData.GPS;``PRN_number = 5;``PRN_index = GPS_rinexData.SatelliteID == PRN_number;``PRN05_GPS_rinexData = GPS_rinexData(PRN_index,:);``[NIST_GPS_week, NIST_GPS_TOW] = utc2gpstime(PRN05_GPS_rinexData.Time);``ephem_time_s = NIST_GPS_week.*604800 + NIST_GPS_TOW;`*% Nan gap the discontinuous*`gap_idx = [false; diff(ephem_time_s) > 1.5*median(diff(ephem_time_s))];``PRN05_GPS_rinexData.C1C(gap_idx) = NaN;``PRN05_GPS_rinexData.S1C(gap_idx) = NaN;``PRN05_GPS_rinexData.C1W(gap_idx) = NaN;``PRN05_GPS_rinexData.D1C(gap_idx) = NaN;`

(b.)

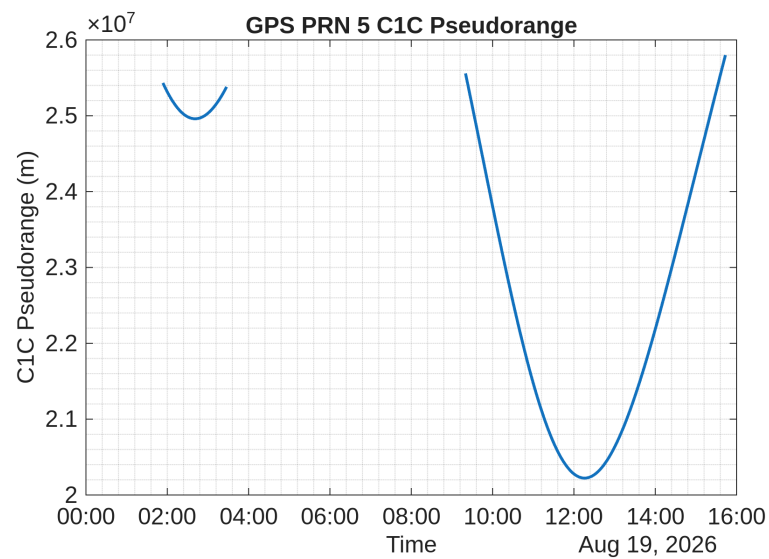


Figure 1: C1C Pseudorange for PRN5

(c.)

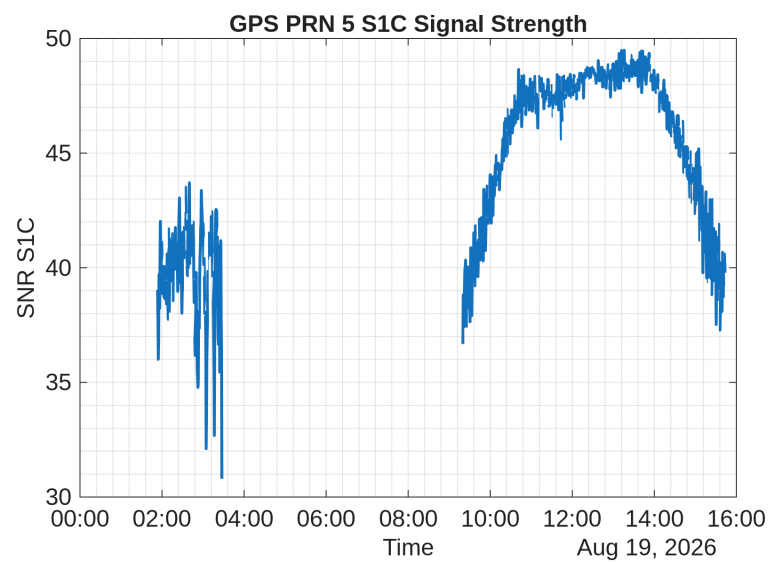


Figure 2: S1C SNR for PRN5

(d.)

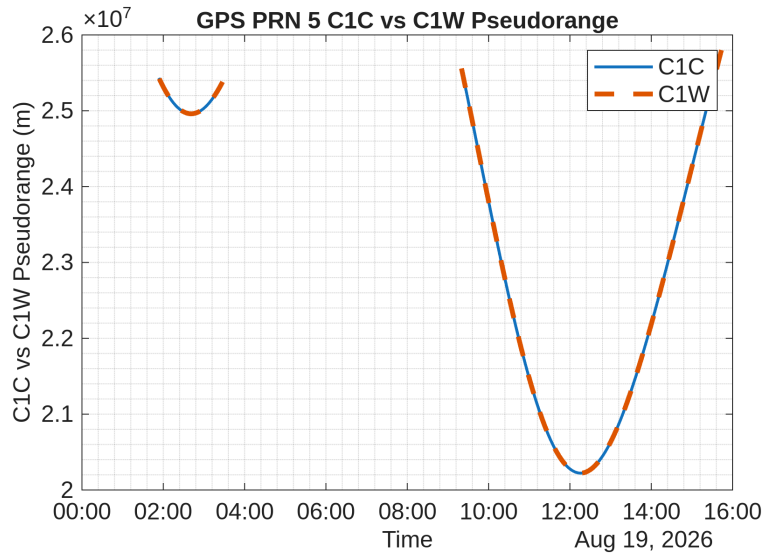


Figure 3: C1C and C1W Pseudorange for PRN5

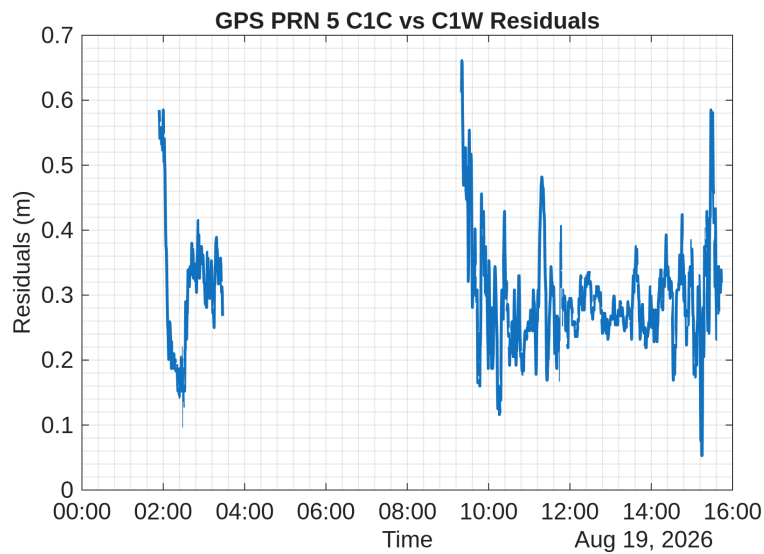


Figure 4: C1C and C1W Residuals for PRN5

When examining the plots for C1C and C1W, we notice that they follow each other trends very well. Their reported pseudoranges deviate very minorly, when examining their residuals there are only differences of up to 0.7m between the two pseudoranges. This is reasonable as C1C and C1W are the C/A code and P code respectively. It is natural that they would be reporting similar numbers. The variations between them are a result from noise and multipathing, which would lead to the very minor variations that can be seen from Figure 4.

Problem 2: GPS Positions Based on Ephemeris Data

(a.)

```
% Use read clean to read in ephermides
clean_GPSbroadcast = read_clean_GPSbroadcast('brdc2310.26n', true);

% Read in sp3 ephem like HW2
sp3 = read_sp3('IGS00PSFIN_20262310000_01D_15M_ORB.SP3');
satPRN = extract_sp3_prn(sp3);

[health,satPos,satVel,satClkCorr] = eph2pvt2025(clean_GPSbroadcast, [satPRN(5).
    week_number satPRN(5).TOW_s], 5);
```

(b.)

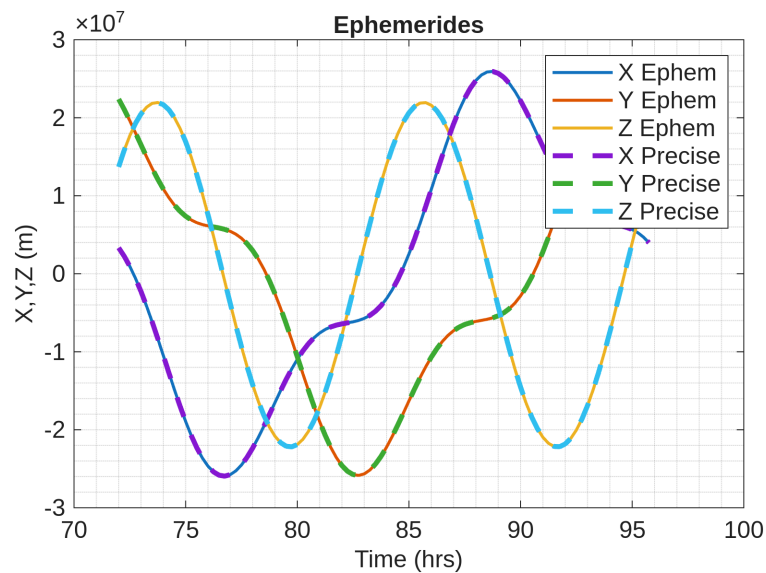


Figure 5: Ephemeris vs Precise

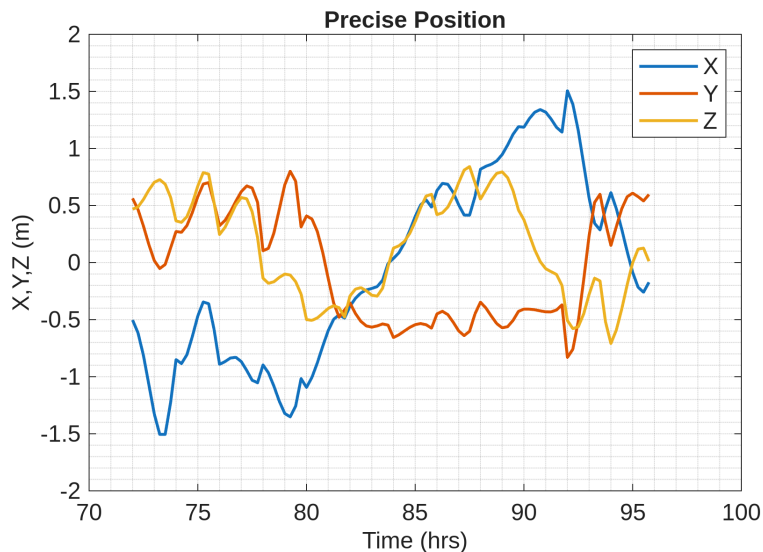


Figure 6: Ephemeris vs Precise

Taking a look at these results, the precise positioning found from SP3 and the ephemeris broadcast in Figure 5, we see that they line up very well, by further using the data obtained by SP3 as truth, we find residuals of the ephemeris broadcast to be up to $\approx 1.5\text{m}$ from Figure 6. Compared to the results from HW2 on the almanac's residuals, which were up to 1 km, this is far more accurate. It shows that the almanac would be useful for checking visibility of satellites in the sky, however poor for positioning. The resulting residuals we observe here can begin to be used for positioning.

(c.)

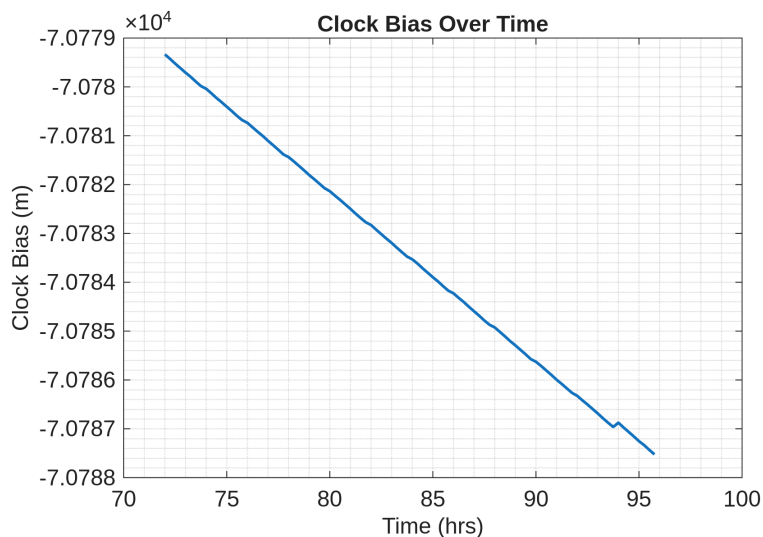


Figure 7: Clock Bias

Problem 3: GPS Expected Range

(a.)

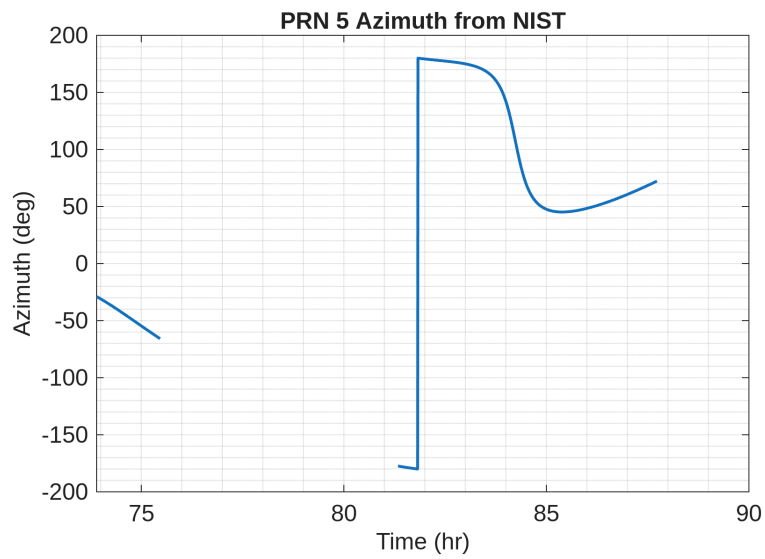


Figure 8: PRN5 Azimuth

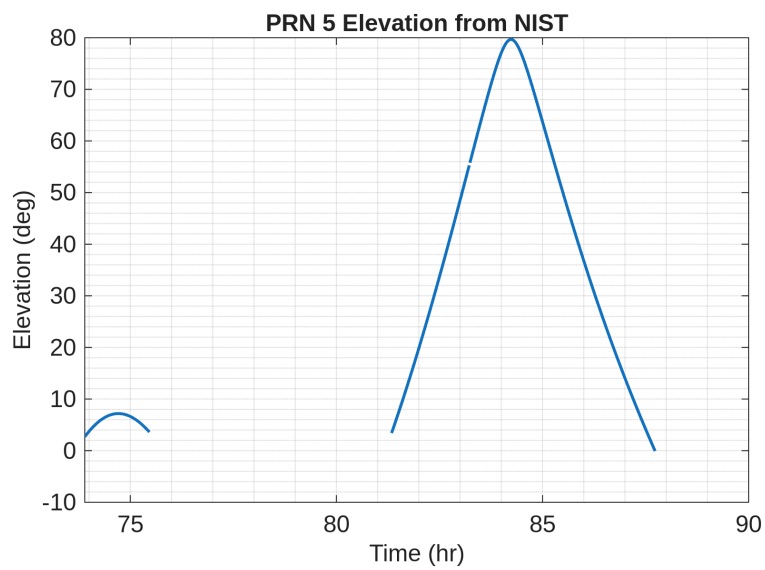


Figure 9: PRN5 Elevation

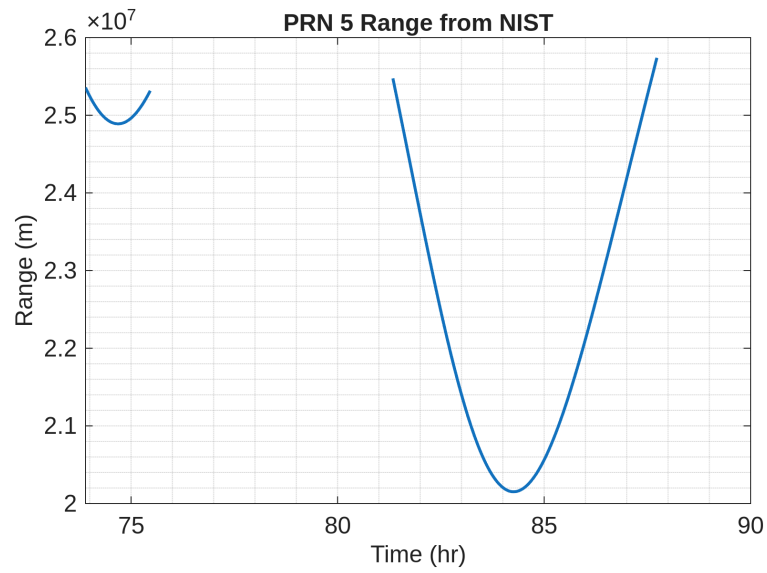


Figure 10: PRN5 Range

(b.)

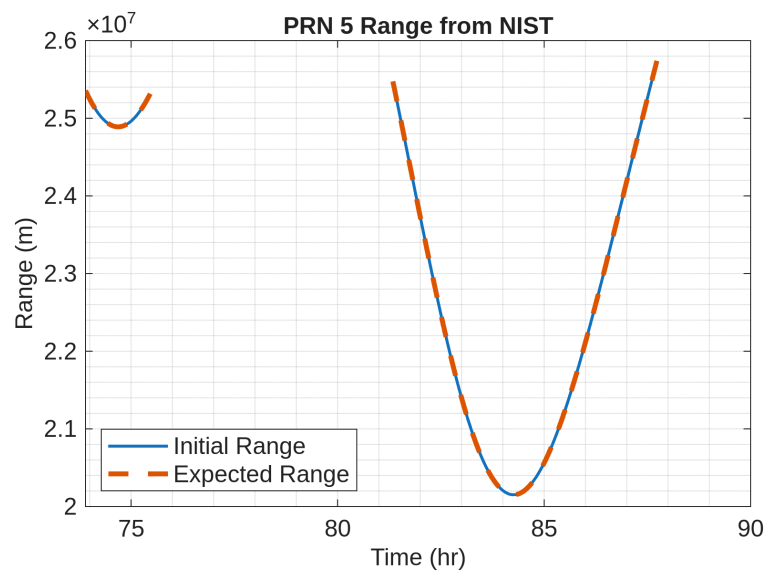


Figure 11: PRN5 Range Initial vs Expected

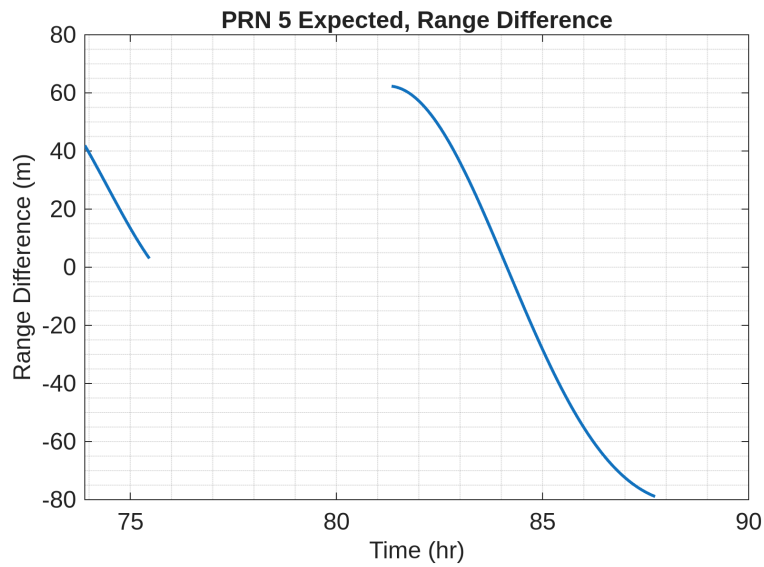


Figure 12: PRN5 Range Initial vs Expected Residual

From here, we can find the largest difference from the expected range and the initial range we obtain from ephemeris to be about 78.9m. This is a rather large deviation between the ranges, showing how there is still correction needed due to the movement of the satellite during the signal travel time causing discrepancies in the initial ranging calculations from the broadcasted ephemeris received from a ground station/gps receiver.

Problem 4: Observed Minus Computed (Expected) Range

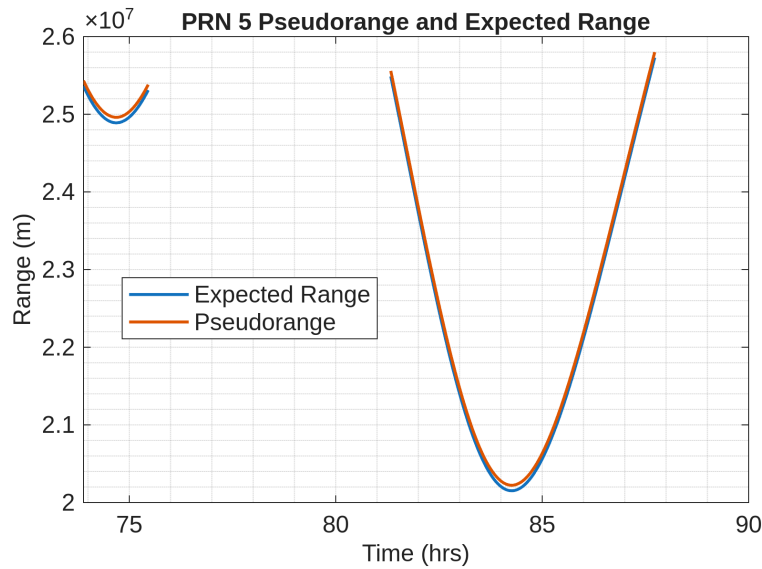


Figure 13: PRN5 C1C Pseudorange vs Expected Range

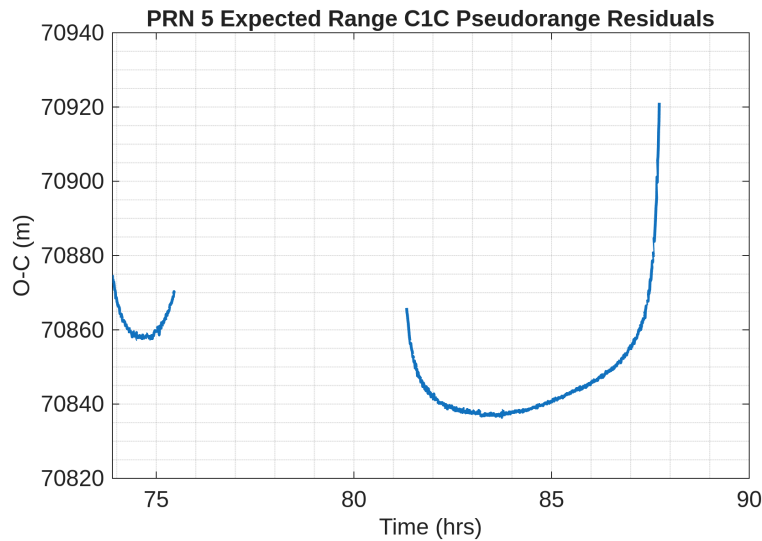


Figure 14: PRN5 C1C Pseudorange vs Expected Range Residuals

We are able to observe the differences between the reported pseudoranges and the expected range calculated previously in Figure 13 and Figure 14. They generally match up, however there are residuals of up to 85 km. The biggest differences between them are likely a result of the correction being done to find the expected range. The ephemeris broadcast, conversion to position, and subsequent correction we performed would be much more accurate than the pseudorange that is being reported. The pseudorange after all is affected by propagation effects in the atmosphere (tropospheric effects, ionospheric effects) which cause these errors observed between the two ranging measurements being reported.

Proof can be shown when examining these results for PRN 11 which is done for problem 5 as well. Examining Figures 15 and 16, we see residuals in the same magnitude as seen in Figures 13 and 14. This means that throughout this process, the difference is not a result from using a different GPS satellite, PRN 11 and PRN 5 are in different satellite blocks and would be in view of NIST at different times through their orbits. The factors that would affect pseudoranging in this case that would not be due to the GPS satellites would be atmospheric effects and the clock errors in the GPS receiver at the ground station. Additional evidence at this being a clock error issue is examining the clock errors plotted from Problem 2, Figure 7. Here we can see an clock bias error of about 70 km. The same magnitude of error that we observe in the psuedorange and expected range residuals.

Problem 5: Repeat for another PRN

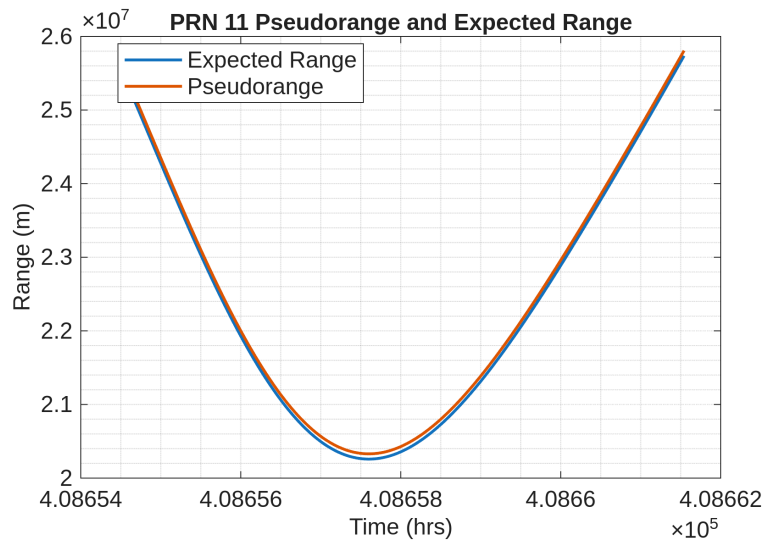


Figure 15: PRN11 C1C Pseudorange vs Expected Range

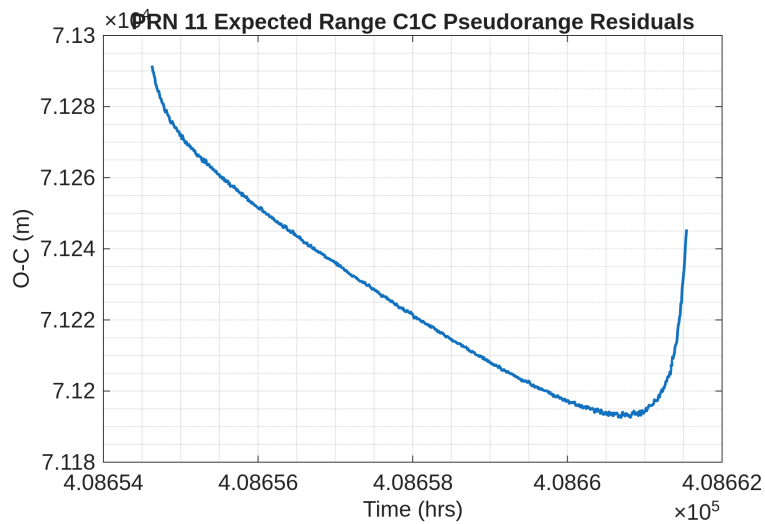


Figure 16: PRN11 C1C Pseudorange vs Expected Range Residuals

Here PRN 11 is chosen, this is in a different GPS block than PRN 5 with notable different visibility patterns in the same time frame for NIST.

```
% ASEN 5090 GPS/GNSS
%% Justin Le
%% HW3 Part 1 Main Script

clc; clear; close all

% Read rinex file

rinexData = rinexread('NIST00USA_R_20262310000_01D_30S_M0.rnx');

GPS_rinexData = rinexData.GPS;

PRN_number = 5;
PRN_index = GPS_rinexData.SatelliteID == PRN_number;
PRN05_GPS_rinexData = GPS_rinexData(PRN_index,:);

[NIST_GPS_week, NIST_GPS_TOW] = utc2gpstime(PRN05_GPS_rinexData.Time);
ephem_time_s = NIST_GPS_week.*604800 + NIST_GPS_TOW;

% Nan gap the discontinuous
gap_idx = [false; diff(ephem_time_s) > 1.5*median(diff(ephem_time_s))];
PRN05_GPS_rinexData.C1C(gap_idx) = NaN;
PRN05_GPS_rinexData.S1C(gap_idx) = NaN;
PRN05_GPS_rinexData.C1W(gap_idx) = NaN;
PRN05_GPS_rinexData.D1C(gap_idx) = NaN;

% hihihihihihihihihihihihihihihihihihihihihihhi

figure()
plot(PRN05_GPS_rinexData.Time, PRN05_GPS_rinexData.C1C, 'LineWidth', 1.5)
xlabel('Time')
ylabel('C1C Pseudorange (m)')
title(sprintf('GPS PRN %d C1C Pseudorange', PRN_number))
grid minor
set(gcf, 'Units', 'inches', 'Position', [0 0 6 4]);
set(findall(gcf, '-property', 'FontSize'), 'FontSize', 12);
exportgraphics(gcf, fullfile('figures', 'HW3_P1_C1C_PRN5.png'), 'Resolution', 300);

figure()
plot(PRN05_GPS_rinexData.Time, PRN05_GPS_rinexData.S1C, 'LineWidth', 1.5)
xlabel('Time')
ylabel('SNR S1C')
title(sprintf('GPS PRN %d S1C Signal Strength', PRN_number))
grid minor
set(gcf, 'Units', 'inches', 'Position', [0 0 6 4]);
set(findall(gcf, '-property', 'FontSize'), 'FontSize', 12);
exportgraphics(gcf, fullfile('figures', 'HW3_P1_S1C_PRN5.png'), 'Resolution', 300);

figure()
plot(PRN05_GPS_rinexData.Time, PRN05_GPS_rinexData.C1W, 'LineWidth', 1.5)
xlabel('Time')
title(sprintf('GPS PRN %d C1W Pseudorange', PRN_number))
```

```
ylabel('C1W')
grid minor
set(gcf, 'Units', 'inches', 'Position', [0 0 6 4]);
set(findall(gcf, '-property', 'FontSize'), 'FontSize', 12);
exportgraphics(gcf, fullfile('figures', 'HW3_P1_C1W_PRN5.png'), 'Resolution', 300);

% Plotting C1W and C1C
figure()
plot(PRN05_GPS_rinexData.Time, PRN05_GPS_rinexData.C1C, 'LineWidth', 1.5)
hold on
plot(PRN05_GPS_rinexData.Time, PRN05_GPS_rinexData.C1W, '--', 'LineWidth', 2.5)
xlabel('Time')
ylabel('C1C vs C1W Pseudorange (m)')
title(sprintf('GPS PRN %d C1C vs C1W Pseudorange', PRN_number))
grid minor
legend('C1C', 'C1W')
set(gcf, 'Units', 'inches', 'Position', [0 0 6 4]);
set(findall(gcf, '-property', 'FontSize'), 'FontSize', 12);
exportgraphics(gcf, fullfile('figures', 'HW3_P1_C1C_C1W_PRN5.png'), 'Resolution', 300);

% Plotting C1W and C1C residuals
figure()
plot(PRN05_GPS_rinexData.Time, PRN05_GPS_rinexData.C1C-PRN05_GPS_rinexData.C1W, 'LineWidth', 1.5)
xlabel('Time')
ylabel('Residuals (m)')
title(sprintf('GPS PRN %d C1C vs C1W Residuals', PRN_number))
grid minor
set(gcf, 'Units', 'inches', 'Position', [0 0 6 4]);
set(findall(gcf, '-property', 'FontSize'), 'FontSize', 12);
exportgraphics(gcf, fullfile('figures', 'HW3_P1_C1C_C1W_PRN5_residuals.png'), 'Resolution', 300);
```

```

%% ASEN 5090 GPS/GNSS
%% Justin Le
%% HW3 Part 2 Main Script
clc;clear;close all

% Use read clean to read in ephermides
clean_GPSbroadcast = read_clean_GPSbroadcast('brdc2310.26n', true);

% Read in sp3 ephem like HW2
sp3 = read_sp3('IGS00PSFIN_20262310000_01D_15M_ORB.SP3');
satPRN = extract_sp3_prn(sp3);

[health,satPos,satVel,satClkCorr] = eph2pvt2025(clean_GPSbroadcast, [satPRN(5).
    week_number satPRN(5).TOW_s], 5);

% Plotting ephemerides
figure()
plot(satPRN(5).TOW_hr, satPos, 'LineWidth', 1.5)
xlabel('Time (hrs)')
ylabel('X,Y,Z (m)')
title('Ephemerides')
grid minor
legend('X','Y','Z')
set(gcf, 'Units', 'inches', 'Position', [0 0 6 4]);
set(findall(gcf, '-property', 'FontSize'), 'FontSize', 12);

% Plotting precise position
figure()
plot(satPRN(5).TOW_hr, [satPRN(5).X_m,satPRN(5).Y_m,satPRN(5).Z_m], 'LineWidth', 1.5)
xlabel('Time (hrs)')
ylabel('X,Y,Z (m)')
title('Precise Position')
grid minor
legend('X','Y','Z')
set(gcf, 'Units', 'inches', 'Position', [0 0 6 4]);
set(findall(gcf, '-property', 'FontSize'), 'FontSize', 12);

% Plotting Precise on top of Ephemeris
figure()
plot(satPRN(5).TOW_hr, satPos, 'LineWidth', 1.5)
hold on
plot(satPRN(5).TOW_hr, [satPRN(5).X_m,satPRN(5).Y_m,satPRN(5).Z_m], '--', 'LineWidth',
    2.5)
xlabel('Time (hrs)')
ylabel('X,Y,Z (m)')
title('Ephemerides')
grid minor
legend('X Ephem','Y Ephem','Z Ephem','X Precise','Y Precise','Z Precise')
set(gcf, 'Units', 'inches', 'Position', [0 0 6 4]);
set(findall(gcf, '-property', 'FontSize'), 'FontSize', 12);
exportgraphics(gcf, fullfile('figures', 'HW3_P2_ephem_precise_PRN5.png'), 'Resolution',
    300);

% Plotting residual

```

```
figure()
plot(satPRN(5).TOW_hr, [satPRN(5).X_m-satPos(:,1), satPRN(5).Y_m-satPos(:,2), satPRN(5).
    Z_m-satPos(:,3)], 'LineWidth', 1.5)
xlabel('Time (hrs)')
ylabel('X,Y,Z (m)')
title('Precise Position')
grid minor
legend('X','Y','Z')
set(gcf, 'Units', 'inches', 'Position', [0 0 6 4]);
set(findall(gcf, '-property', 'FontSize'), 'FontSize', 12);
exportgraphics(gcf, fullfile('figures', 'HW3_P2_xyz_residual_PRN5.png'), 'Resolution',
    300);

% c. Get satellite clock bias

% Get bias from ephemeris data

figure()
plot(satPRN(5).TOW_hr, satClkCorr, 'LineWidth', 1.5)
xlabel('Time (hrs)')
ylabel('Clock Bias (m)')
grid minor
title('Clock Bias Over Time')
set(gcf, 'Units', 'inches', 'Position', [0 0 6 4]);
set(findall(gcf, '-property', 'FontSize'), 'FontSize', 12);
exportgraphics(gcf, fullfile('figures', 'HW3_P2_clock_bias_PRN5.png'), 'Resolution', 300)
;
```

```

%% ASEN 5090 GPS/GNSS
%% Justin Le
%% HW3 Part 3 Main Script
clc;clear;close all

% Reading in rinex
rinexData = rinexread('NIST00USA_R_20262310000_01D_30S_M0.rnx');
rinexDataGPS = rinexData.GPS;

% Read in ephemeris
clean_GPSbroadcast = read_clean_GPSbroadcast('brdc2310.26n', true);

PRN_number = 5;
PRN_index = rinexDataGPS.SatelliteID == PRN_number;
PRN05_GPS_rinexData = rinexDataGPS(PRN_index,:);

[NIST_GPS_week, NIST_GPS_TOW] = utc2gpstime(PRN05_GPS_rinexData.Time);

[health,satPos,satVel,satClkCorr] = eph2pyt2025(clean_GPSbroadcast, [NIST_GPS_week
    NIST_GPS_TOW], 5);

NIST_ECEF = [-1288398.567 -4721696.932 4078625.350];

ephem_time_s = NIST_GPS_week.*604800 + NIST_GPS_TOW;
ephem_time_hr = NIST_GPS_TOW/3600;

[AZ_NIST, EL_NIST, RANGE_NIST] = compute_azelrange(NIST_ECEF, satPos);

% Nan gap data due to discontinuity.
gap_idx = [false; diff(ephem_time_s) > 1.5*median(diff(ephem_time_s))];
AZ_NIST(gap_idx) = NaN;
EL_NIST(gap_idx) = NaN;
RANGE_NIST(gap_idx) = NaN;

figure()
plot(ephem_time_hr, AZ_NIST, 'LineWidth', 1.5)
grid minor
xlabel('Time (hr)')
ylabel('Azimuth (deg)')
title(sprintf('PRN %d Azimuth from NIST', PRN_number))
set(gcf, 'Units', 'inches', 'Position', [0 0 6 4]);
set(findall(gcf, '-property', 'FontSize'), 'FontSize', 12);
exportgraphics(gcf, fullfile('figures', 'HW3_P3_azimuth_PRN5.png'), 'Resolution', 300);

figure()
plot(ephem_time_hr, EL_NIST, 'LineWidth', 1.5)
grid minor
xlabel('Time (hr)')
ylabel('Elevation (deg)')
title(sprintf('PRN %d Elevation from NIST', PRN_number))
set(gcf, 'Units', 'inches', 'Position', [0 0 6 4]);
set(findall(gcf, '-property', 'FontSize'), 'FontSize', 12);
exportgraphics(gcf, fullfile('figures', 'HW3_P3_elevation_PRN5.png'), 'Resolution', 300);

```

```

figure()
plot(ephem_time_hr, RANGE_NIST, 'LineWidth', 1.5)
grid minor
xlabel('Time (hr)')
ylabel('Range (m)')
title(sprintf('PRN %d Range from NIST', PRN_number))
set(gcf, 'Units', 'inches', 'Position', [0 0 6 4]);
set(findall(gcf, '-property', 'FontSize'), 'FontSize', 12);
exportgraphics(gcf, fullfile('figures', 'HW3_P3_range_PRN5.png'), 'Resolution', 300);

% Expected range corrected for light-time and Earth rotation
R_expected = compute_expected_range(clean_GPSbroadcast, NIST_GPS_week, NIST_GPS_TOW,
    PRN_number, NIST_ECEF);

R_expected(gap_idx) = NaN;

% Add the expected range to the previous range plot from ephem
figure()
plot(ephem_time_hr, RANGE_NIST, 'LineWidth', 1.5)
grid minor
xlabel('Time (hr)')
ylabel('Range (m)')
title(sprintf('PRN %d Range from NIST', PRN_number))
hold on
plot(ephem_time_hr, R_expected, '--', 'LineWidth', 2.5)
legend('Initial Range', 'Expected Range', 'Location', 'southwest')
set(gcf, 'Units', 'inches', 'Position', [0 0 6 4]);
set(findall(gcf, '-property', 'FontSize'), 'FontSize', 12);
exportgraphics(gcf, fullfile('figures', 'HW3_P3_initial_range_vs_expected_PRN5.png'), '
    Resolution', 300);

% Plot the difference between the expected initial ranging
range_diff = R_expected - RANGE_NIST;
max_range_diff = max(abs(range_diff))

figure()
plot(ephem_time_hr, range_diff, 'LineWidth', 1.5)
grid minor
xlabel('Time (hr)')
ylabel('Range Difference (m)')
title(sprintf('PRN %d Expected, Range Difference', PRN_number))
set(gcf, 'Units', 'inches', 'Position', [0 0 6 4]);
set(findall(gcf, '-property', 'FontSize'), 'FontSize', 12);
exportgraphics(gcf, fullfile('figures', 'HW3_P3_range_difference_PRN5.png'), 'Resolution'
    , 300);

```

```

%% ASEN 5090 GPS/GNSS
%% Justin Le
%% HW3 Part 4 Main Script
clc;clear;close all

% Reading in rinex
rinexData = rinexread('NIST00USA_R_20262310000_01D_30S_M0.rnx');
rinexDataGPS = rinexData.GPS;

% Read in ephemeris
clean_GPSbroadcast = read_clean_GPSbroadcast('brdc2310.26n', true);

PRN_number = 5;
PRN_index = rinexDataGPS.SatelliteID == PRN_number;
PRN05_GPS_rinexData = rinexDataGPS(PRN_index,:);

[NIST_GPS_week, NIST_GPS_TOW] = utc2gpstime(PRN05_GPS_rinexData.Time);

NIST_ECEF = [-1288398.567 -4721696.932 4078625.350];

ephem_time_s = NIST_GPS_week.*604800 + NIST_GPS_TOW;
ephem_time_hr = NIST_GPS_TOW/3600;

gap_idx = [false; diff(ephem_time_s) > 1.5*median(diff(ephem_time_s))];

% Expected range corrected for light-time and Earth rotation
R_expected = compute_expected_range(clean_GPSbroadcast, NIST_GPS_week, NIST_GPS_TOW,
    PRN_number, NIST_ECEF);
R_expected(gap_idx) = NaN;
PRN05_GPS_rinexData.C1C(gap_idx) = NaN;

% a. Plot psuedorange and expected ranges and then residuals
figure()
plot(ephem_time_hr, R_expected, 'LineWidth', 1.5)
hold on
plot(ephem_time_hr, PRN05_GPS_rinexData.C1C, 'LineWidth', 1.5)
xlabel('Time (hrs)')
ylabel('Range (m)')
grid minor
legend('Expected Range', 'Pseudorange', 'Location', 'best')
title(sprintf('PRN %d Pseudorange and Expected Range', PRN_number))
set(gcf, 'Units', 'inches', 'Position', [0 0 6 4]);
set(findall(gcf, '-property', 'FontSize'), 'FontSize', 12);
exportgraphics(gcf, fullfile('figures', 'HW3_P4_pseudorange_vs_expected_PRN5.png'), '
    Resolution', 300);

figure()
plot(ephem_time_hr, PRN05_GPS_rinexData.C1C-R_expected, 'LineWidth', 1.5)
xlabel('Time (hrs)')
ylabel('O-C (m)')
grid minor
title(sprintf('PRN %d Expected Range C1C Pseudorange Residuals', PRN_number))
ax = gca; ax.YAxis.Exponent = 0;
set(gcf, 'Units', 'inches', 'Position', [0 0 6 4]);

```

```
set(findall(gcf, '-property', 'FontSize'), 'FontSize', 12);  
exportgraphics(gcf, fullfile('figures', 'HW3_P4_residuals_PRN5.png'), 'Resolution', 300);
```

```

%% ASEN 5090 GPS/GNSS
%% Justin Le
%% HW3 Part 5 Main Script
clc;clear;close all

% Reading in rinex
rinexData = rinexread('NIST00USA_R_20262310000_01D_30S_M0.rnx');
rinexDataGPS = rinexData.GPS;

% Read in ephemeris
clean_GPSbroadcast = read_clean_GPSbroadcast('brdc2310.26n', true);

PRN_number = 11;
PRN_index = rinexDataGPS.SatelliteID == PRN_number;
PRN11_GPS_rinexData = rinexDataGPS(PRN_index,:);

[NIST_GPS_week, NIST_GPS_TOW] = utc2gpstime(PRN11_GPS_rinexData.Time);

NIST_ECEF = [-1288398.567 -4721696.932 4078625.350];

ephem_time_s = NIST_GPS_week.*604800 + NIST_GPS_TOW;
ephem_time_hr = ephem_time_s/3600;

gap_idx = [false; diff(ephem_time_s) > 1.5*median(diff(ephem_time_s))];

% Expected range corrected for light-time and Earth rotation
R_expected = compute_expected_range(clean_GPSbroadcast, NIST_GPS_week, NIST_GPS_TOW,
    PRN_number, NIST_ECEF);
R_expected(gap_idx) = NaN;

% a. Plot psuedorange and expected ranges, and then residuals
figure()
plot(ephem_time_hr, R_expected, 'LineWidth', 1.5)
hold on
plot(ephem_time_hr, PRN11_GPS_rinexData.C1C, 'LineWidth', 1.5)
xlabel('Time (hrs)')
ylabel('Range (m)')
grid minor
legend('Expected Range', 'Pseudorange', 'Location', 'best')
title(sprintf('PRN %d Pseudorange and Expected Range', PRN_number))
set(gcf, 'Units', 'inches', 'Position', [0 0 6 4]);
set(findall(gcf, '-property', 'FontSize'), 'FontSize', 12);
exportgraphics(gcf, fullfile('figures', 'HW3_P5_pseudorange_vs_expected_PRN11.png'), '
    Resolution', 300);

figure()
plot(ephem_time_hr, PRN11_GPS_rinexData.C1C-R_expected, 'LineWidth', 1.5)
xlabel('Time (hrs)')
ylabel('O-C (m)')
grid minor
title(sprintf('PRN %d Expected Range C1C Pseudorange Residuals', PRN_number))
set(gcf, 'Units', 'inches', 'Position', [0 0 6 4]);
set(findall(gcf, '-property', 'FontSize'), 'FontSize', 12);
exportgraphics(gcf, fullfile('figures', 'HW3_P5_residuals_PRN11.png'), 'Resolution', 300)

```

;

```

function R_expected = compute_expected_range(ephemeris, week, tow, PRN_number,
    receiver_ECEF)
% Computes expected range by iteration

% Constants for the alg
c = 2.99792458e8; % speed of light, m/s
wE = 7.2921151467e-5; % Earth rotation rate, rad/s

% Step 1: Satellite ECEF position at Tr initially.
Tr = tow + week*604800;
[~, satPos, ~, ~] = eph2pvt2025(ephemeris, [week tow], PRN_number);

% Step 2: Geometric range using NIST_ECEF and satPos at Tr
% Reuse azel range for convinience
[~, ~, R] = compute_azelrange(receiver_ECEF, satPos);

% Iterate until range converges
% Step 3: initial time of transmission
Tt = Tr - (R/c);
R_expected = R;

for iter = 1:5
    % Step 3: time of transmission in gps week number and TOW
    Tt_week = floor(Tt/604800);
    Tt_tow = Tt - Tt_week*604800;

    % Step 4: satellite ECEF position at Tt
    [~, satPos_Tt, ~, ~] = eph2pvt2025(ephemeris, [Tt_week Tt_tow], PRN_number);

    % Step 5: rotate sat position into the ECEF frame
    % Sat position from receiver
    x = satPos_Tt(:,1);
    y = satPos_Tt(:,2);
    z = satPos_Tt(:,3);

    phi = wE .* (Tr - Tt);
    x_ECEF = x.*cos(phi) + y.*sin(phi);
    y_ECEF = -x.*sin(phi) + y.*cos(phi);

    satPos_Tt_ECEF = [x_ECEF, y_ECEF, z];

    % Step 6: recompute geometric range with the rotated sat position
    [~, ~, R_new] = compute_azelrange(receiver_ECEF, satPos_Tt_ECEF);

    % Convergeance condition
    if max(abs(R_new - R_expected)) < 1e-4
        R_expected = R_new;
        break
    end
    R_expected = R_new;
    Tt = Tr - R_expected/c; % Step 3 repeat
end

end
end

```

```
function [AZ, EL, RANGE] = compute_azelrange(userECEF, satECEF)
% satECEF is ECEF vector of sat
% userECEF is ECEF vector of the user

% Calculate LLA from userECEF
lla = ecef2lla(userECEF,'WGS84');
ref_lat = lla(1);
ref_lon = lla(2);
altitude = lla(3);
% Take difference of ECEF vectors, conver to ENU depending on ref angle
r_ECEF = satECEF-userECEF;

% Calculating range.
RANGE = sqrt(sum(r_ECEF.^2,2));

% Converting to ENU frame
r_ENU = (ECEF2ENU(ref_lat,ref_lon)*r_ECEF')';
x_E = r_ENU(:,1);
x_N = r_ENU(:,2);
x_U = r_ENU(:,3);

% Calculating azimuth and elevation
AZ = atan2d(x_E,x_N);
EL = asind(x_U./sqrt(sum(r_ENU.^2,2)));

end
```

```
function [week_number, tow] = utc2gpstime(utc_datetime)
% Converts utc format to gps accounting for leapseconds as well

    gps_epoch = datetime(1980,1,6,0,0,0,'TimeZone','UTC');
    gps_datetime = datetime(utc_datetime,'TimeZone','UTC');

    seconds_since_gps_epoch = seconds(gps_datetime - gps_epoch);

    week_number = floor(seconds_since_gps_epoch/604800);
    tow = seconds_since_gps_epoch - week_number*604800;
end
```

```

function satPRN = extract_sp3_prn(sp3)
% Extract and filter SP3 data for any single PRN.
week_number = sp3(:,1);
TOW_s = sp3(:,2);
PRN = sp3(:,3);
X_m = sp3(:,4)*1000;
Y_m = sp3(:,5)*1000;
Z_m = sp3(:,6)*1000;
clock_bias_us = sp3(:,7);
constellation = sp3(:,8);
time_epoch_s = TOW_s;

%Filtering the PRN for all PRNs

for prn_index = 1:32

prn_select = PRN == prn_index;

week_number_prn = week_number(prn_select);
TOW_s_prn = TOW_s(prn_select);
TOW_hr_prn = TOW_s_prn/3600;
time_epoch_s_prn = time_epoch_s(prn_select);
time_epoch_hr_prn = time_epoch_s_prn;
X_m_prn = X_m(prn_select);
Y_m_prn = Y_m(prn_select);
Z_m_prn = Z_m(prn_select);
clock_bias_us_prn = clock_bias_us(prn_select);
constellation_prn = constellation(prn_select);
PRN_prn = PRN(prn_select);

% Insert into a struct to output for use of use

satPRN(prn_index) = struct(...
    'week_number', week_number_prn, ...
    'TOW_s', TOW_s_prn, ...
    'TOW_hr', TOW_hr_prn, ...
    'time_epoch_s', time_epoch_s_prn, ...
    'time_epoch_hr', time_epoch_hr_prn, ...
    'X_m', X_m_prn, ...
    'Y_m', Y_m_prn, ...
    'Z_m', Z_m_prn, ...
    'clock_bias_us', clock_bias_us_prn, ...
    'constellation', constellation_prn, ...
    'PRN', PRN_prn);

end

end

```